Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build publish a reproducible local deployment manifest for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Deploy CertificateNFT locally, issue ID1 and check owner/code. Open deployment.json. Reconcile chainId/address/compiler/ABI/runtimeKeccak/sourceSha256 with the current source and get_code. Re-run default script: explain independent chain state, then use --rpc for persistent Anvil state.

Expected assertions: Nonempty runtime code; owner correct; manifest hashes represent current source and chain code.

Extension prompt

Compare two runs where source changes; show source and runtime hashes differ, keeping old evidence explicitly separate. Return the exact changed source lines and positive/negative tests.